

Operating Systems

Dr. Mostafa Elgendy

Computer Science Department,

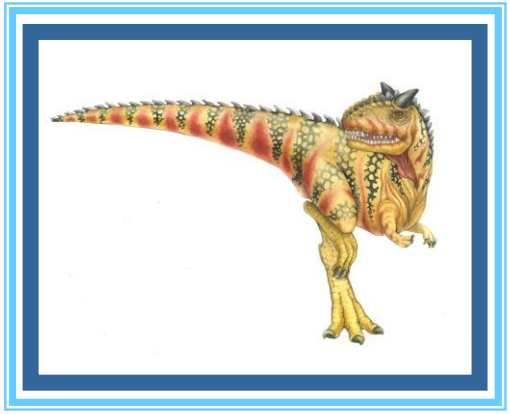
Faculty of Computers and Artificial Intelligence

Benha University

2022



Chapter 3
Processes- Part 2



Silberschatz, Galvin and Gagne ©2018



Outline

- Interprocess Communication
- Shared memory
- Message passing





Interprocess Communication





Interprocess Communication

- Processes within a system may be independent or cooperating
- A process is independent
  - Doesn't share data with any other processes executing in the system.
- A process is cooperating
  - Affect or be affected by the other processes executing in the system.





Interprocess Communication

- Reasons for cooperating processes:
  - **Information sharing:** several applications may be interested in the same piece of information (copying and pasting)
  - **Computation speedup:** need a particular task to run faster, we break it into subtasks which will be executed in parallel.
  - **Modularity:** want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.





Interprocess Communication

- Cooperating processes need **interprocess communication (IPC)**
- Two models of IPC
  - **Shared memory:** a region of memory that is shared by the cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region.
  - **Message passing:** a communication takes place by means of messages exchanged between the cooperating processes.





Interprocess Communication

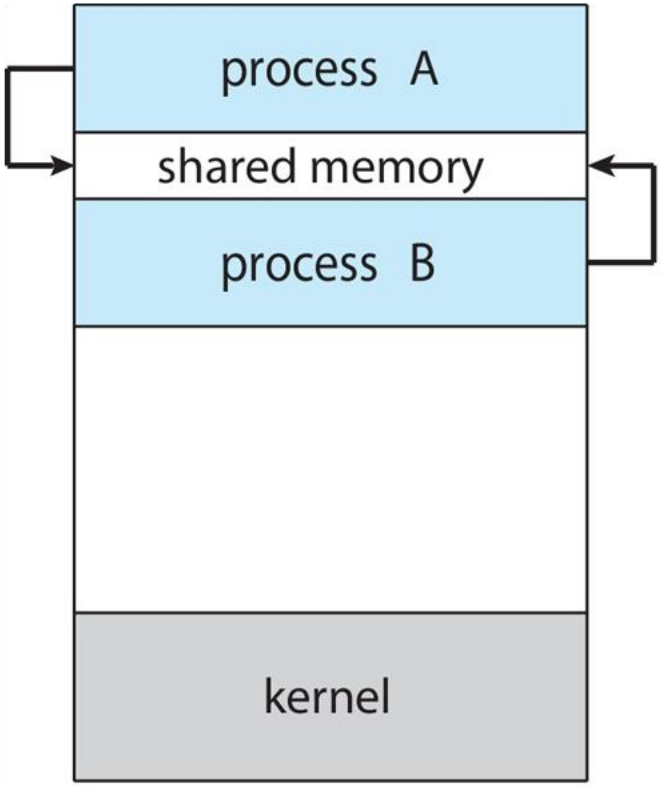
- Both models are common in operating systems.
- Message passing is useful
  - For exchanging smaller amounts of data, because no conflicts need be avoided.
  - Message passing is also easier to implement in a distributed system than shared memory.
- Shared memory
  - Faster than message passing, as message-passing systems are typically implemented using system calls and thus require the more time-consuming task of kernel intervention.





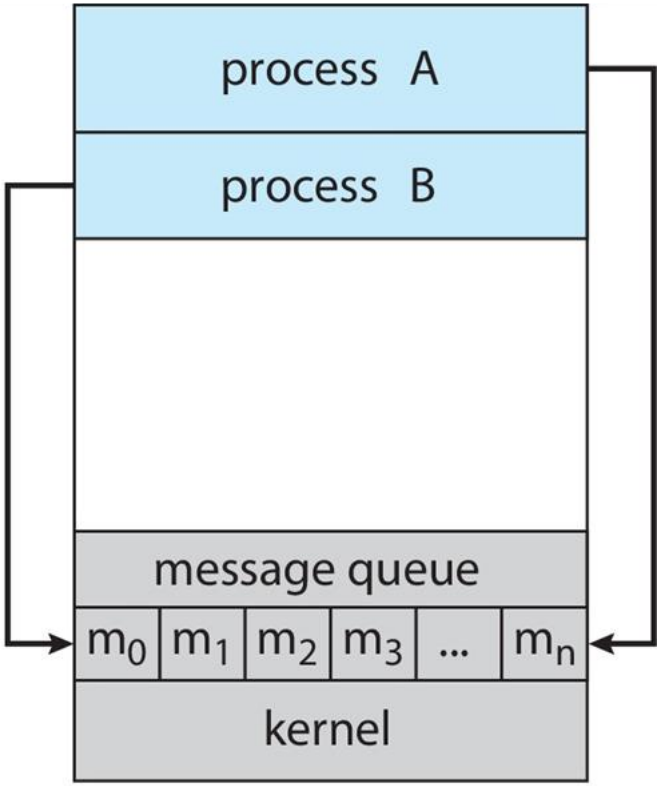
Communications Models

(a) Shared memory.



(a)

(b) Message passing.



(b)





IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The operating system tries to prevent one process from accessing another process's memory.
  - Shared memory requires that two or more processes agree to remove this restriction.
- They can then exchange information by reading and writing data in the shared areas.
  - The data and the location are determined by these processes and are not under the operating system's control.
- The processes are also responsible for ensuring that they are not writing to the same location simultaneously.





Producer-Consumer Problem

- Paradigm for cooperating processes:
  - producer process produces information that is consumed by a consumer process
- We generally think of a server as a producer and a client as a consumer.
  - For example, a web server produces (that is, provides) web content such as HTML files and images.
  - These content are consumed (that is, read) by the client web browser requesting the resource.
- One solution to the producer–consumer problem uses shared memory.





Producer-Consumer Problem (Cont.)

- For the producer and consumer processes to run concurrently
  - A buffer of items that can be filled by the producer and emptied by the consumer.
  - This buffer will reside in a region of memory that is shared by the producer and consumer processes.
- The producer and consumer must be synchronized
  - the consumer does not try to consume an item that has not yet been produced.





Producer-Consumer Problem (Cont.)

- Two types of buffers can be used:
  - **Unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **Bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume





IPC – Message Passing

- A mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space.
- Useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.
- For example, an Internet chat program
  - Participants communicate by exchanging messages
- IPC facility provides two operations:
  - send(message)
  - receive(message)
- The message size is either fixed or variable





Message Passing (Cont.)

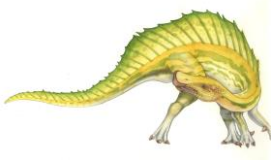
- If processes P and Q wish to communicate, they need:
  - Establish a **communication link** between them
  - Exchange messages via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?





Implementation of Communication Link

- Physical:
  - Hardware bus
  - Network
- Logical:
  - Direct or indirect
  - Synchronous or asynchronous
  - Automatic or explicit buffering





Direct Communication

- Processes must name each other explicitly:
  - **send** (P, message) – send a message to process P
  - **receive**(Q, message) – receive a message from process Q
- Properties of communication link
  - Links are established automatically
  - A link is associated with exactly two processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional but is usually bi-directional.





Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
  - Each mailbox has a unique id.
  - Processes can communicate only if they share a mailbox.
- Properties of communication link
  - A link is established between a pair of processes only if both members of the pair have a shared mailbox.
  - A link may be associated with more than two processes.
  - Each pair of processes may share several communication links.
  - Link may be unidirectional or bi-directional.





Indirect Communication (Cont.)

- Operations
  - Create a new mailbox (port)
  - Send and receive messages through mailbox
  - Delete a mailbox
- Primitives are defined as:
  - **send**(A, message) – send a message to mailbox A
  - **receive**(A, message) – receive a message from mailbox A





Indirect Communication (Cont.)

- Mailbox sharing
  - ***P1***, ***P2***, and ***P3*** share mailbox ***A***.
  - ***P1***, sends; ***P2*** and ***P3*** receive
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver (that is, either ***P2*** or ***P3***, but not both, will receive the message).
  - Sender is notified who received the message.





Synchronization

- Message passing may be either blocking or non-blocking.
- **Blocking** is considered **synchronous**
  - **Blocking send** - the sender is blocked until the message is received.
  - **Blocking receive** - the receiver is blocked until a message is available.





Synchronization (Cont.)

- Message passing may be either blocking or non-blocking.
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** - the sender sends the message and continue
  - **Non-blocking receive** - the receiver receives:
    - A valid message, or
    - Null message.





Buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
  - Zero capacity – no messages are queued on a link.

    Sender must wait for receiver (rendezvous)
  - Bounded capacity – finite length of n messages

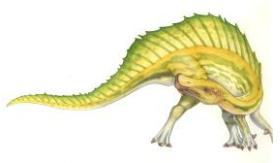
    Sender must wait if link full
  - Unbounded capacity – infinite length

    Sender never waits





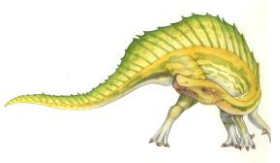
Communications in Client-Server Systems





Communications in Client-Server Systems

- Sockets.
- Remote Procedure Calls.





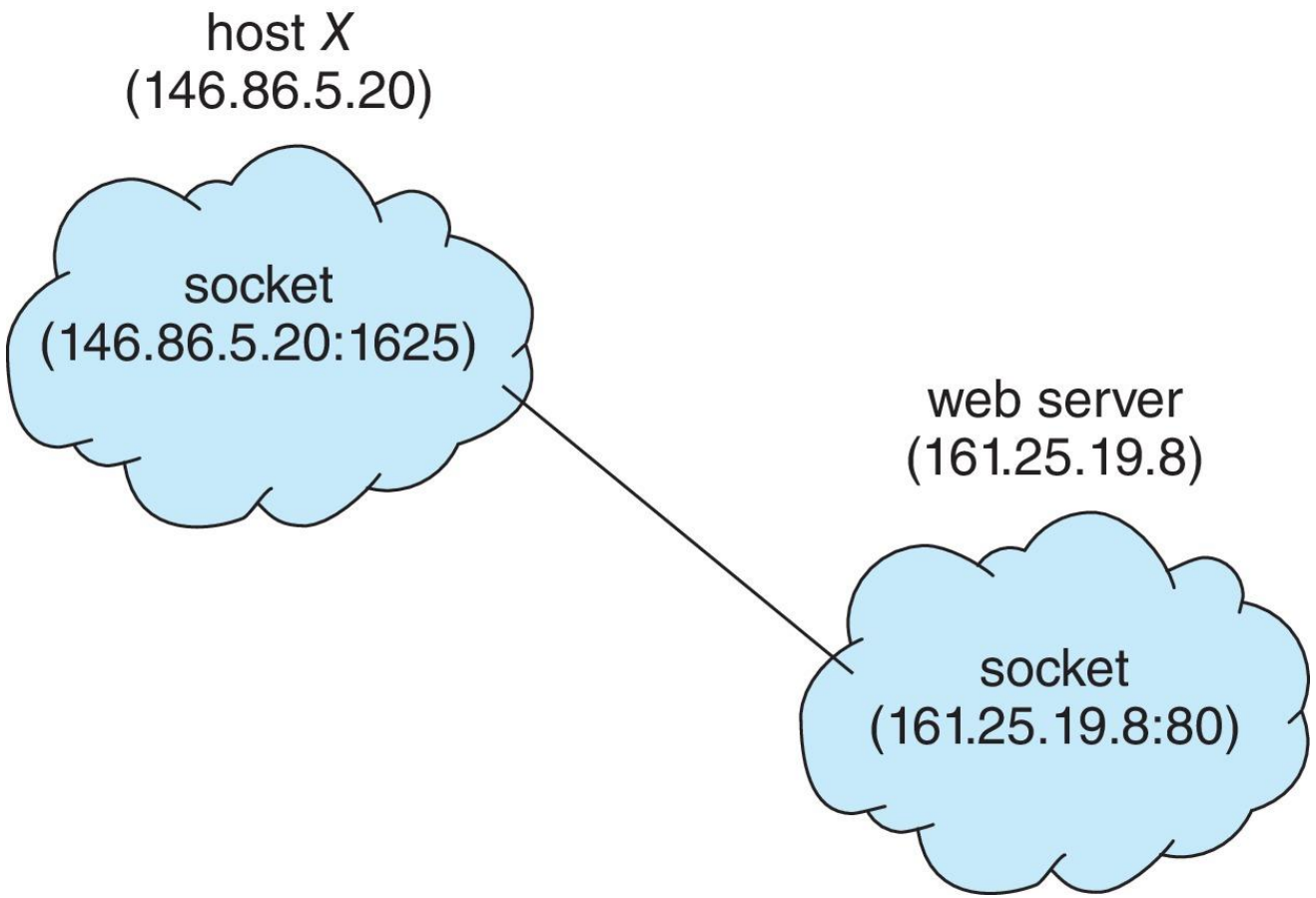
Sockets

- A **socket** is defined as an endpoint for communication.
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket 161.25.19.8:1625 refers to port 1625 on host 161.25.19.8
- Communication consists between a pair of sockets
- All ports below 1024 are well known, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running





Socket Communication







Sockets in Java - Server

- Three types of sockets
  - Connection-oriented (TCP)
  - Connectionless (UDP)
  - MulticastSocket class– data can be sent to multiple recipients.
- Consider this “Date” server in Java:

```
import java.net.*;
import java.io.*;

public class DateServer
{
   public static void main(String[] args) {
      try {
         ServerSocket sock = new ServerSocket(6013);

         /* now listen for connections */
         while (true) {
            Socket client = sock.accept();

            PrintWriter pout = new
             PrintWriter(client.getOutputStream(), true);

            /* write the Date to the socket */
            pout.println(new java.util.Date().toString());

            /* close the socket and resume */
            /* listening for connections */
            client.close();
         }
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```





Sockets in Java - Server

- Three types of sockets
  - Connection-oriented (TCP)
  - Connectionless (UDP)
  - MulticastSocket class– data can be sent to multiple recipients.
- Consider this “Date” server in Java:

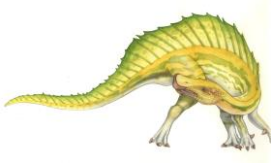
```
import java.net.*;
import java.io.*;

public class DateClient
{
   public static void main(String[] args) {
      try {
         /* make connection to server socket */
         Socket sock = new Socket("127.0.0.1",6013);

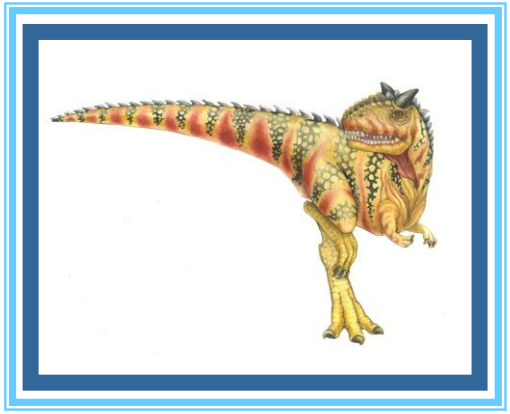
         InputStream in = sock.getInputStream();
         BufferedReader bin = new
            BufferedReader(new InputStreamReader(in));

         /* read the date from the socket */
         String line;
         while ( (line = bin.readLine()) != null)
            System.out.println(line);

         /* close the socket connection*/
         sock.close();
      }
      catch (IOException ioe) {
         System.err.println(ioe);
      }
   }
}
```



End of Chapter 3



Silberschatz, Galvin and Gagne ©2018



THANK YOU

